Desarrollo Multiplataforma – 2º DAW

Práctica Lección 3

Introducción a las uniones entre tablas

Alumno/a: Alejandro Molina Hernández Grupo: 2º DAW

Fecha: Viernes 6 de noviembre de 2026

OBJETIVO

Consultar información relacionada de dos tablas y aplicar filtros, ordenaciones y agrupaciones para obtener resultados útiles.

ANTES DE EMPEZAR

Necesitas acceder a Oracle APEX y disponer del archivo Crimes.sql, que contiene las tablas y los datos de la práctica. El archivo debe facilitarlo el profesor.

Conceptos que vas a utilizar

NATURAL JOIN: Relaciona tablas mediante las columnas comunes con el mismo nombre.

WHERE: Filtra los registros que cumplen unas condiciones.

ORDER BY: Ordena los resultados. Usa ASC (ascendente) o DESC (descendente).

GROUP BY y COUNT: Agrupa registros y cuenta cuántos hay en cada grupo.

ENTREGA

Completa las consultas, ejecútalas en APEX y añade una captura de pantalla de los resultados en los espacios de evidencias. Conserva el código SQL utilizado.

Importante: los valores de texto deben escribirse entre comillas simples.

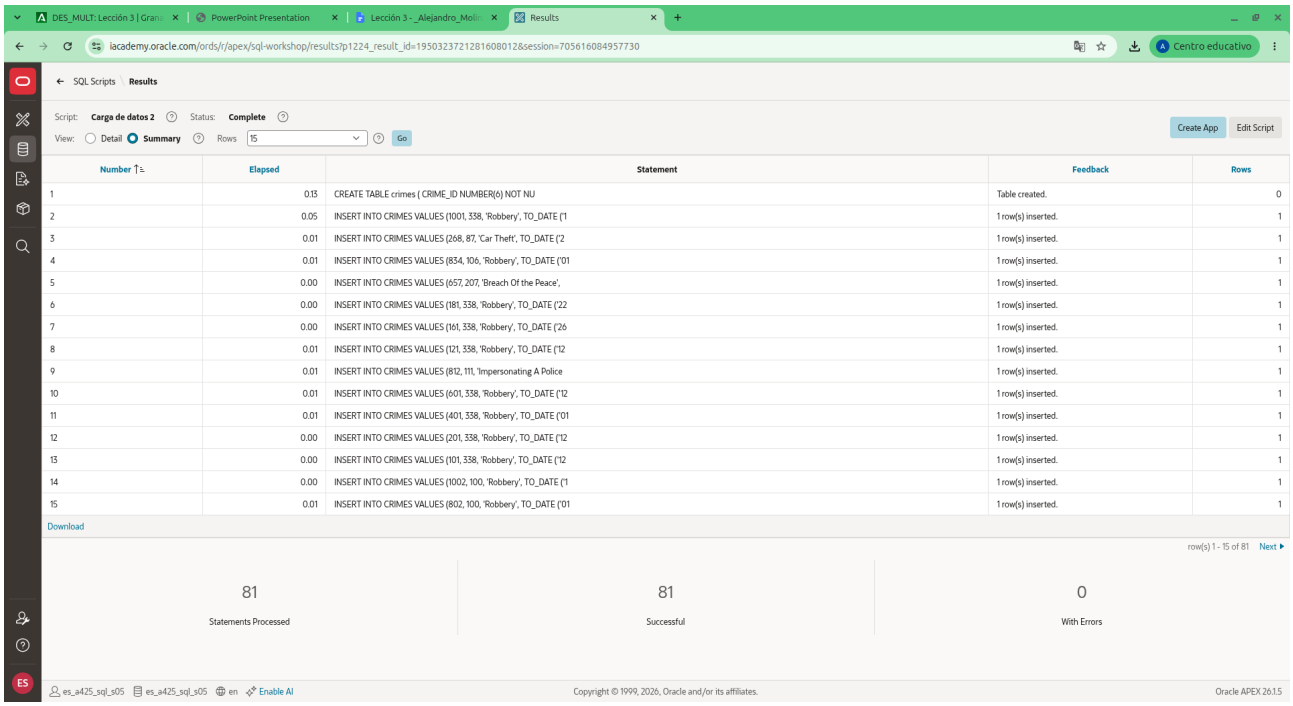
1. Preparación de la base de datos

Carga el script Crimes.sql desde SQL Workshop → SQL Scripts.

Pasos

1. Pulsa Upload y selecciona el archivo Crimes.sql.
2. Pulsa Upload para incorporarlo.
3. Ejecuta el script y confirma la operación (Run Now).
4. Comprueba que la ejecución finaliza correctamente.

EVIDENCIA 1 · Captura de la ejecución correcta del script



SQL Scripts | Results

Script: Carga de datos 2 Status: Complete

View: Detail Summary Rows 15 Go

Create App Edit Script

Number	Elapsed	Statement	Feedback	Rows
1	0.13	CREATE TABLE crimes (CRIME_ID NUMBER(6) NOT NU	Table created.	0
2	0.05	INSERT INTO CRIMES VALUES (1001, 338, 'Robbery', TO_DATE ('1	1 row(s) inserted.	1
3	0.01	INSERT INTO CRIMES VALUES (268, 87, 'Car Theft', TO_DATE ('2	1 row(s) inserted.	1
4	0.01	INSERT INTO CRIMES VALUES (834, 106, 'Robbery', TO_DATE ('01	1 row(s) inserted.	1
5	0.00	INSERT INTO CRIMES VALUES (657, 207, 'Breach Of the Peace',	1 row(s) inserted.	1
6	0.00	INSERT INTO CRIMES VALUES (181, 338, 'Robbery', TO_DATE ('22	1 row(s) inserted.	1
7	0.00	INSERT INTO CRIMES VALUES (161, 338, 'Robbery', TO_DATE ('26	1 row(s) inserted.	1
8	0.01	INSERT INTO CRIMES VALUES (121, 338, 'Robbery', TO_DATE ('12	1 row(s) inserted.	1
9	0.01	INSERT INTO CRIMES VALUES (812, 111, 'Impersonating A Police	1 row(s) inserted.	1
10	0.01	INSERT INTO CRIMES VALUES (601, 338, 'Robbery', TO_DATE ('12	1 row(s) inserted.	1
11	0.01	INSERT INTO CRIMES VALUES (401, 338, 'Robbery', TO_DATE ('01	1 row(s) inserted.	1
12	0.00	INSERT INTO CRIMES VALUES (201, 338, 'Robbery', TO_DATE ('12	1 row(s) inserted.	1
13	0.00	INSERT INTO CRIMES VALUES (101, 338, 'Robbery', TO_DATE ('12	1 row(s) inserted.	1
14	0.00	INSERT INTO CRIMES VALUES (1002, 100, 'Robbery', TO_DATE ('1	1 row(s) inserted.	1
15	0.01	INSERT INTO CRIMES VALUES (802, 100, 'Robbery', TO_DATE ('01	1 row(s) inserted.	1

Download

row(s) 1 - 15 of 81 Next

81 Statements Processed

81 Successful

0 With Errors

Copyright © 1999, 2026, Oracle and/or its affiliates. Oracle APEX 26.1.5

2. Primera consulta con NATURAL JOIN

La base de datos contiene dos tablas principales: SUSPECTS (información de los sospechosos) y CRIMES (historial de delitos). Ambas comparten el campo SUSPECT_ID.

Tarea

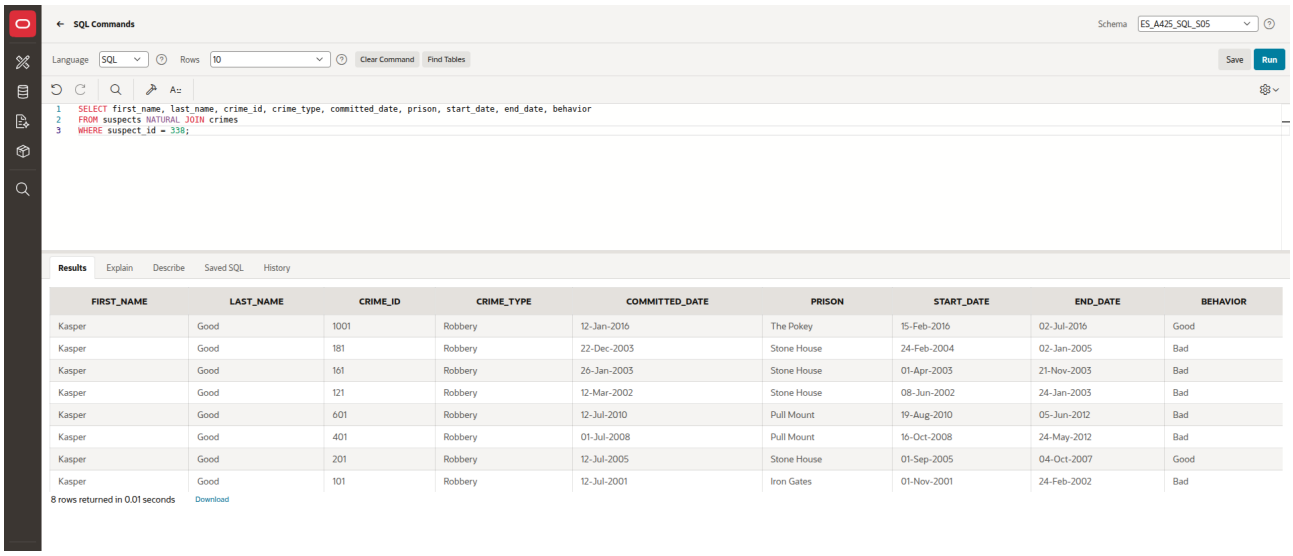
Escribe una consulta que muestre el nombre y apellidos del sospechoso, el identificador y tipo de delito, la fecha de comisión y la prisión. Muestra únicamente los delitos del sospechoso cuyo SUSPECT_ID sea 338.

Pista: utiliza SELECT, FROM suspects NATURAL JOIN crimes y WHERE.

CONSULTA SQL

```
SELECT first_name, last_name, crime_id, crime_type, committed_date, prison, start_date, end_date, behavior
FROM suspects NATURAL JOIN crimes
WHERE suspect_id = 338;
```

EVIDENCIA 2 · Captura de la consulta y de sus resultados



3. Filtrar y ordenar resultados

Realiza las dos consultas siguientes. Recuerda que puedes filtrar utilizando campos de cualquiera de las dos tablas.

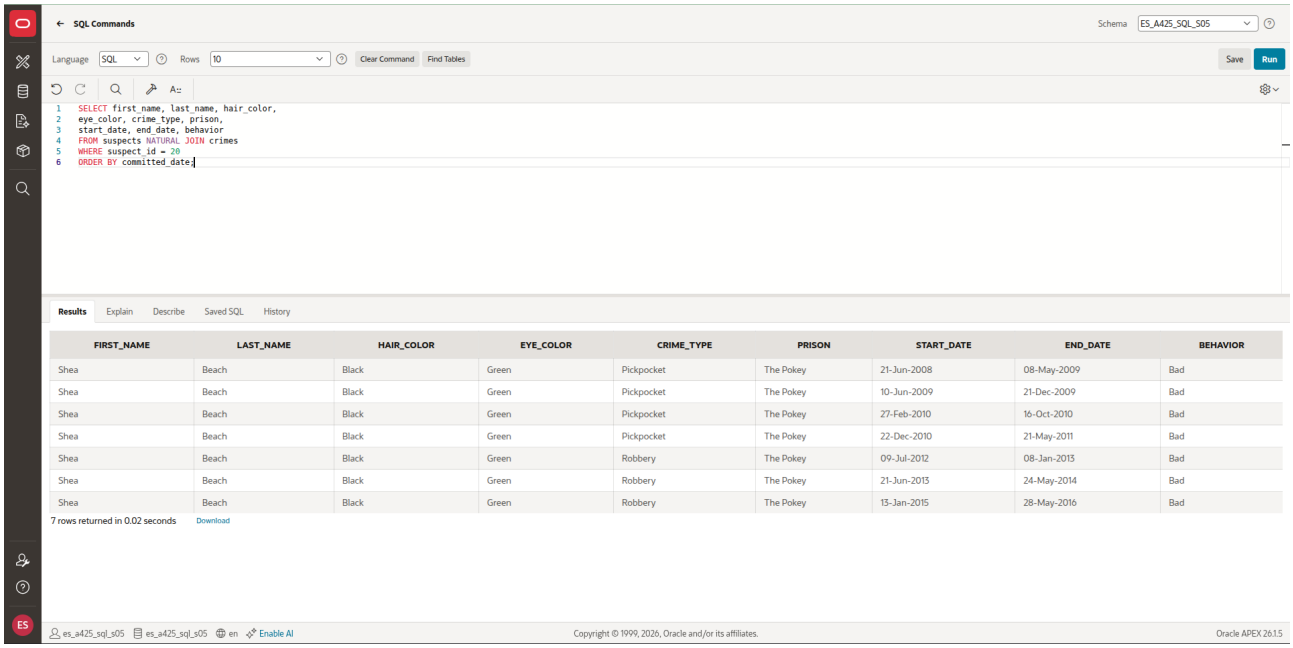
Tarea A · Historial de un sospechoso

Muestra los delitos del sospechoso con SUSPECT_ID = 20. Deben aparecer: nombre, apellidos, color de pelo, color de ojos, tipo de delito, prisión, fecha de inicio y fin de condena y comportamiento. Ordena por la fecha de comisión del delito, de más antiguo a más reciente.

CONSULTA SQL A

```
SELECT first_name, last_name, hair_color,
eye_color, crime_type, prison,
start_date, end_date, behavior
FROM suspects NATURAL JOIN crimes
WHERE suspect_id = 20
ORDER BY committed_date;
```

EVIDENCIA 3 · Resultados de la consulta A



Tarea B · Búsqueda por características

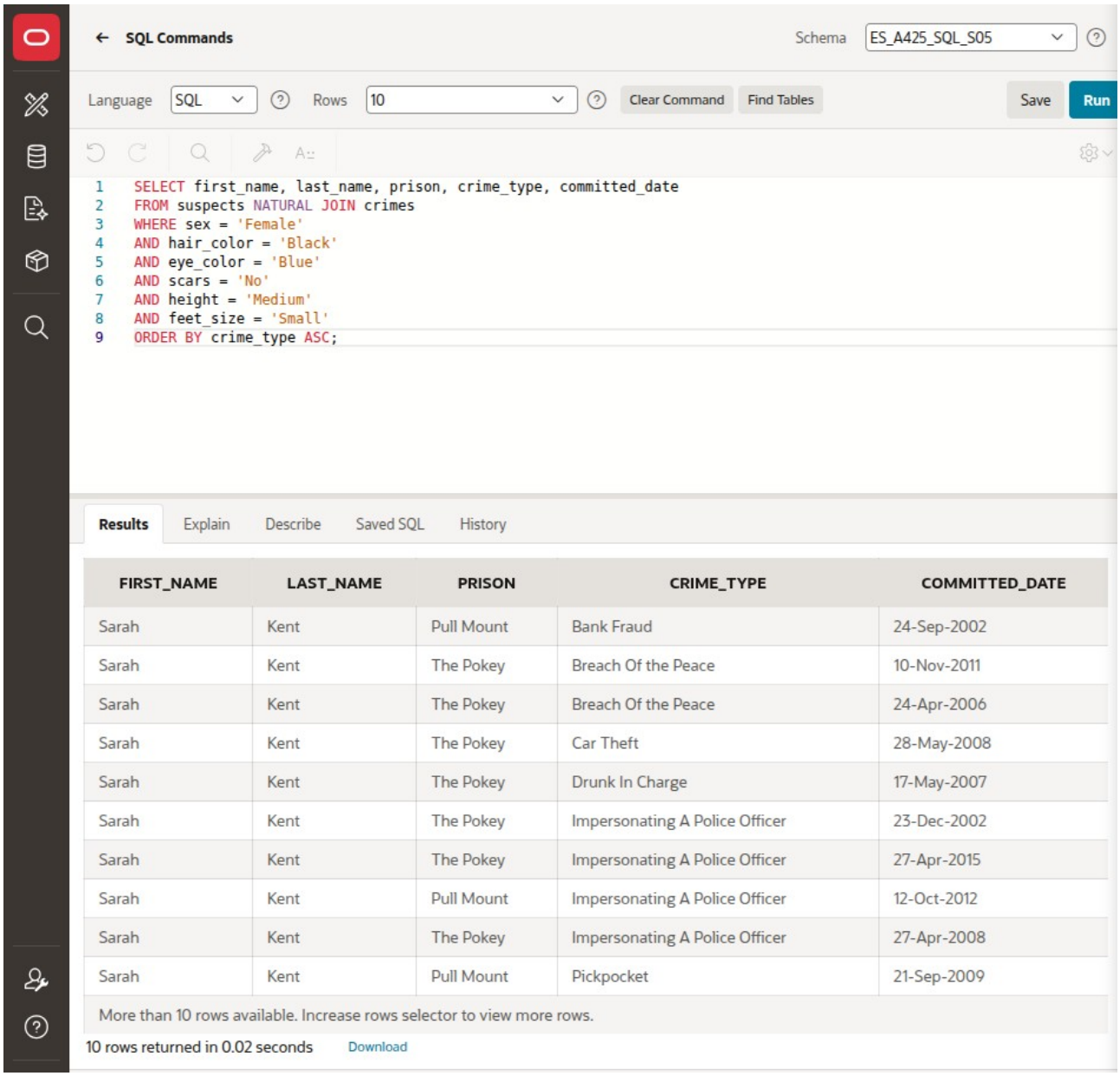
Localiza a la sospechosa que cumple estas características: mujer, pelo negro, ojos azules, altura media, pies pequeños y sin cicatrices. Después, muestra su nombre y apellidos, la prisión y el tipo de delito de su historial. Ordena alfabéticamente por tipo de delito.

Pista: utiliza los campos SEX, HAIR_COLOR, EYE_COLOR, HEIGHT, FEET_SIZE y SCARS.

CONSULTA SQL B

```
SELECT first_name, last_name, prison, crime_type, committed_date
FROM suspects NATURAL JOIN crimes
WHERE sex = 'Female' AND hair_color = 'Black' AND eye_color = 'Blue' AND scars = 'No' AND height = 'Medium' AND feet_size = 'Small' ORDER BY crime_type ASC;
```

EVIDENCIA 4 · Resultados de la consulta B



4. Agrupar y contar con GROUP BY y COUNT

Queremos conocer cuántos delitos de cada tipo ha cometido un sospechoso.

Tarea

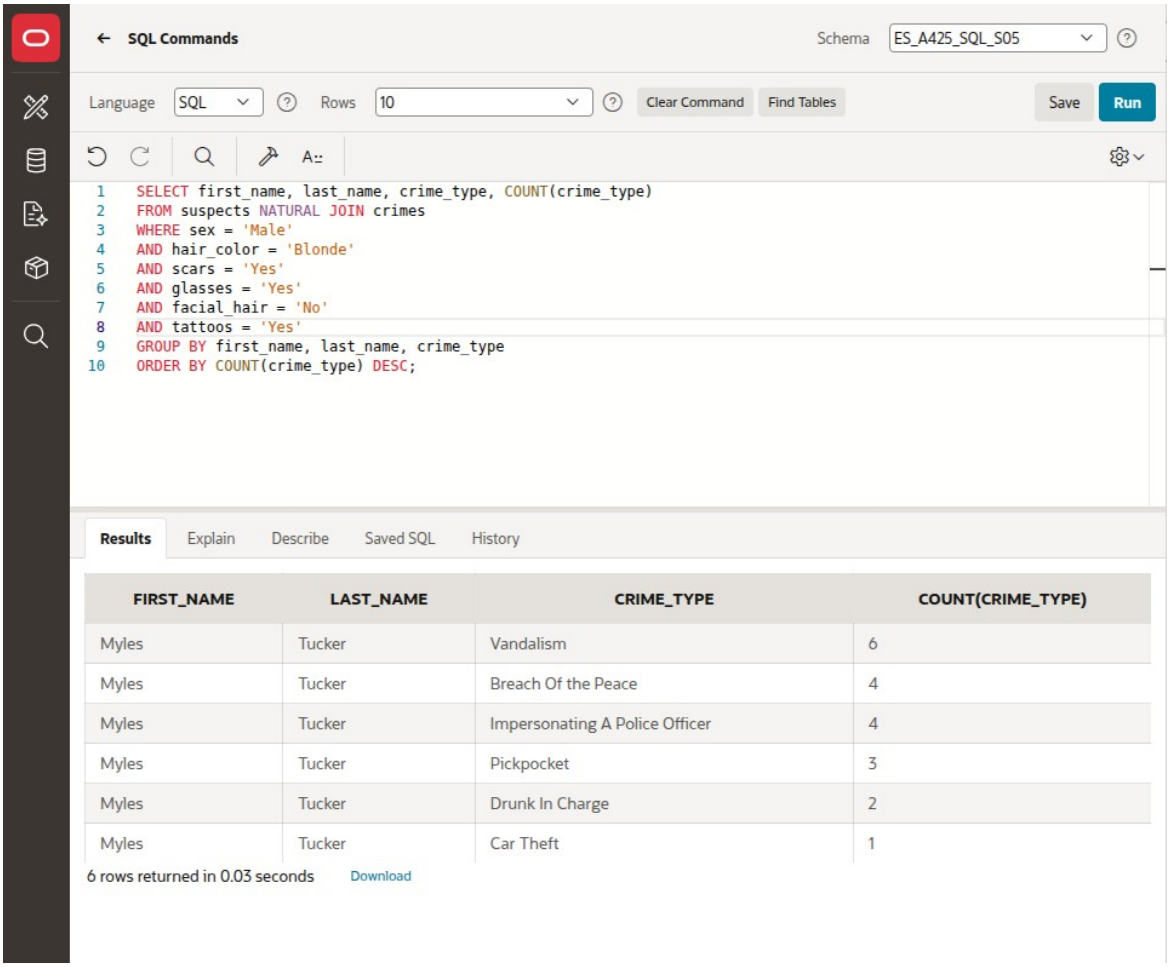
Utiliza la información del sospechoso que cumple estas características: varón, pelo rubio, gafas, sin vello facial, con cicatrices y tatuajes. Realiza una consulta que muestre su nombre y apellidos, el tipo de delito y el número de delitos de cada tipo. Ordena los resultados de mayor a menor número de delitos.

Pista: combina NATURAL JOIN, WHERE, COUNT(crime_type), GROUP BY y ORDER BY COUNT(crime_type) DESC.

CONSULTA SQL

```
SELECT first_name, last_name, crime_type, COUNT(crime_type)
FROM suspects NATURAL JOIN crimes
WHERE sex = 'Male'
AND hair_color = 'Blonde'
AND scars = 'Yes'
AND glasses = 'Yes'
AND facial_hair = 'No'
AND tattoos = 'Yes'
GROUP BY first_name, last_name, crime_type
ORDER BY COUNT(crime_type) DESC;
```

EVIDENCIA 5 · Captura de la consulta y del recuento obtenido



5. Reflexión final

a) ¿Para qué sirve NATURAL JOIN en esta práctica?

Sirve para poder interconectar las características de los sospechosos con sus crímenes. Por ejemplo: alguien a hecho vandalismo y solo se sabe que es rubio, pues con NATURAL JOIN puedes buscar a sospechosos rubios con el crimen cometido, en este caso vandalismo.

b) ¿Qué diferencia hay entre ORDER BY y GROUP BY?

ORDER BY sirve para ordenar los resultados de las consultas por un parámetro. Por ejemplo: Ordenar a los sospechosos por nombre. Y GROUP BY sirve para ordenar por campo. Por ejemplo: Agrupar a todos lo que no tienen cicatrices primero y después a los que si tienen.

c) ¿Qué información aporta COUNT?

Proporciona un recuento de cuantos (por ejemplo) crímenes tiene cada sospechoso. Por ejemplo: si un sospechoso tiene 6 delitos, al poner COUNT (crime_type) se puede saber cuantos delitos de ese tipo ha hecho el sospechoso.

COMPROBACIÓN DE ENTREGA

☐ Script ejecutado ☐ Consultas SQL incluidas ☐ Evidencias capturadas ☐ Reflexión final